

Experiment-5B

Aim: Write a program to develop a single page application using RESTful APIs .

Description:

1. Setup Project

- **express** → Web framework
- **mongoose** → MongoDB ORM
- **body-parser** → Parse JSON request bodies
- **cors** → Allow cross-origin requests
- **nodemon** → Auto-reload server

2. Set up MongoDBnp

- You can use **MongoDB Atlas (cloud)** or **local MongoDB**.
- Create a database (e.g., mydatabase) and a collection (e.g., users).
- Get the connection string (e.g., mongodb://localhost:27017/mydatabase for local, or the Atlas URI for cloud).

3. Create the Server

Program:

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const cors = require('cors');
const app = express();
const PORT = 5000;
// ===== Middleware =====
app.use(cors());
app.use(bodyParser.json());
// ===== MongoDB Connection =====
mongoose.connect('mongodb://127.0.0.1:27017/mydatabase', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log("MongoDB connected"))
.catch(err => console.error(err));
// ===== Mongoose Schema & Model =====
const userSchema = new mongoose.Schema({
  name: { type: String, required: true }
});
const User = mongoose.model('User', userSchema);
// ===== RESTful API Routes =====
// GET all users
app.get('/api/users', async (req, res) => {
```

```
try {
  const users = await User.find();
  res.json(users);
} catch (err) {
  res.status(500).json({ message: err.message });
}
});
// GET user by ID
app.get('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ message: "User not found" });
    res.json(user);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
// POST new user
app.post('/api/users', async (req, res) => {
  const user = new User({ name: req.body.name });
  try {
    const newUser = await user.save();
    res.status(201).json(newUser);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
// PUT update user
app.put('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ message: "User not found" });
    user.name = req.body.name;
    const updatedUser = await user.save();
    res.json(updatedUser);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

```
// DELETE user
app.delete('/api/users/:id', async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ message: "User not found" });
    await user.remove();
    res.json({ message: "User deleted" });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
// ===== Start Server =====
app.listen(PORT, () => console.log(`Server running on http://localhost:\${PORT}`));
```

Package.json:

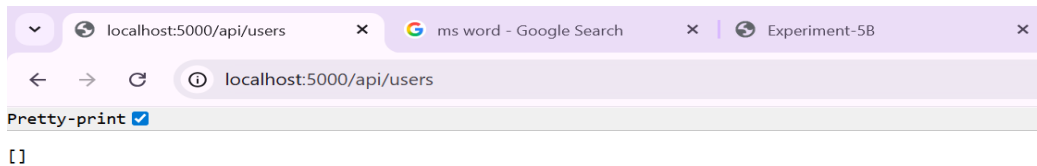
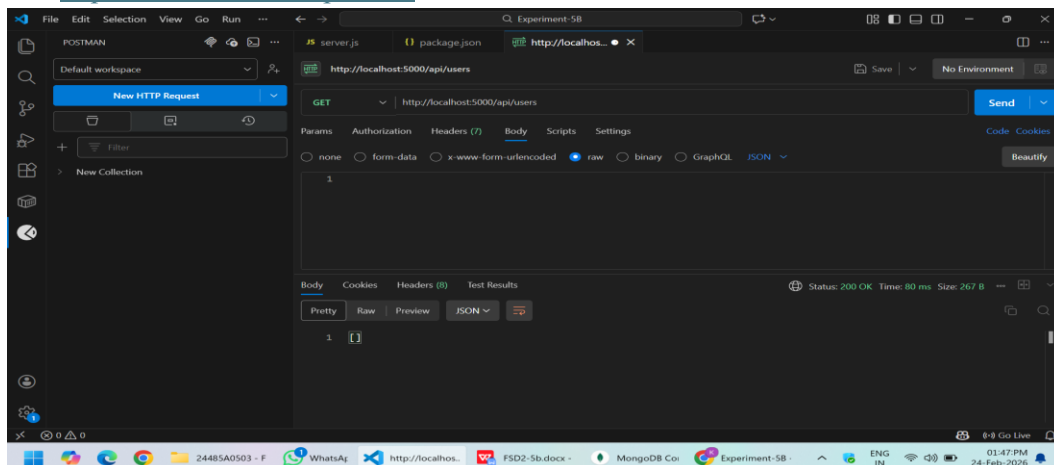
```
"scripts": {
  "start": "nodemon server.js"
}
```

5. Test the API

You can test with Postman or curl:

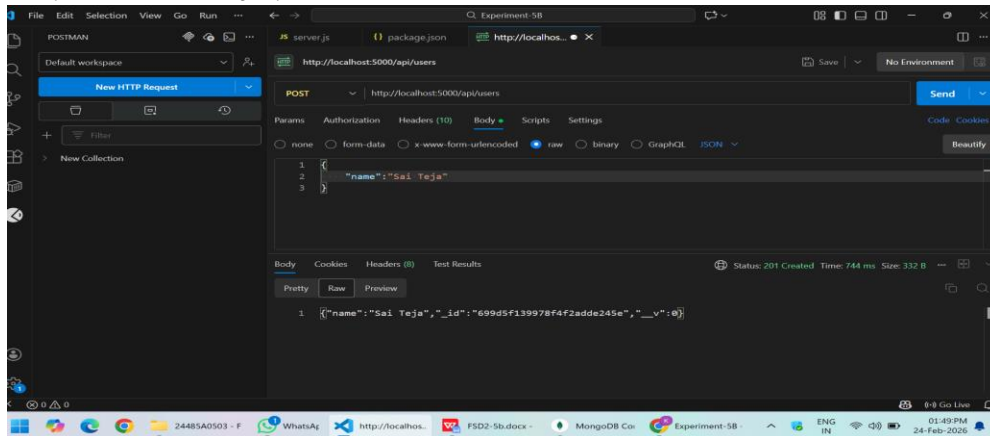
1. GET all users

curl <http://localhost:5000/api/users>

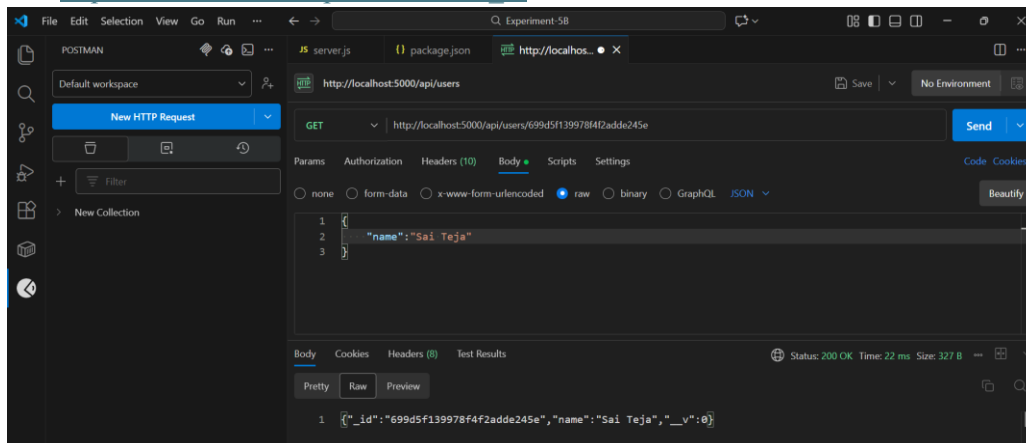


POST a new user

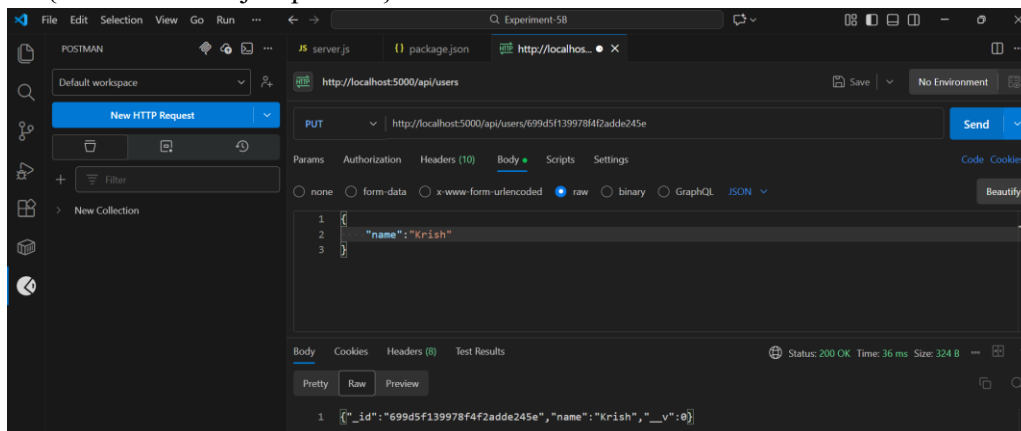
```
curl -X POST http://localhost:5000/api/users \
-H "Content-Type: application/json" \
-d '{"name":"Sai Teja"}'
```

**GET user by ID**

```
curl http://localhost:5000/api/users/<user_id>
```

**PUT update user**

```
curl -X PUT http://localhost:5000/api/users/<user_id> \
-H "Content-Type: application/json" \
-d '{"name":"Sai Teja Updated"}'
```



DELETE user

curl -X DELETE http://localhost:5000/api/users/<user_id>

